

ORCHESTRIX

AI-Powered Workflow Automation Platform

End-to-End Technical Documentation — FastAPI Edition

Complete implementation blueprint for a visual workflow automation platform using Next.js, React Flow, FastAPI, PostgreSQL, Redis, Celery and a Python workflow engine.

Component	Decision
Frontend	Next.js + React + TypeScript + React Flow
Backend	FastAPI + Python
Database	PostgreSQL + SQLAlchemy + Alembic
Queue	Redis + Celery
Workers	Python Celery workers
Real-time	FastAPI WebSockets
Storage	S3-compatible storage / R2
Deployment	Docker + GitHub Actions

Core rule: FastAPI handles requests; Celery workers execute long-running workflows. The API must never block while a workflow runs.

1. Executive Summary

Orchestrix is an n8n-style visual automation platform. Users create directed graphs of trigger, action, logic, data, integration and AI nodes. Workflows can run manually, through webhooks or on schedules.

The FastAPI architecture separates the control plane from execution. PostgreSQL is the durable source of truth. Redis is the queue transport. Celery workers execute workflows. React Flow provides the visual editor.

Primary objectives

- Visual drag-and-drop workflow builder
- Durable workflow versions and execution history
- Asynchronous background execution
- Secure credential management
- Webhooks and schedules
- Modular Python node system
- AI-ready architecture
- Docker-based local and production deployment

2. Scope

Area	MVP	Later
Auth	Register/login/protected APIs	OAuth/SSO/SCIM
Editor	Canvas, nodes, connections, config	Collaboration/comments
Triggers	Manual/webhook/schedule	Integration event triggers
Nodes	HTTP, Set, IF, Switch, Delay	Merge, loops, sub-workflows
Execution	Celery, status, logs, retry	Priority/rate-limit queues
Integrations	HTTP + PostgreSQL/Slack/Gmail	Large integration catalog
AI	Optional LLM node	Agents, RAG, workflow generation

3. Functional Requirements

FR-01 Authentication

Users can register, log in, log out and access only authorized workspaces.

FR-02 Workflow management

Create, edit, save, activate, deactivate, duplicate and delete workflows. Drafts and published versions must be distinguishable.

FR-03 Visual editor

Drag nodes, configure them, connect ports, move/zoom/pan, undo/redo and validate the graph.

FR-04 Execution

Create an execution record, enqueue a Celery job, execute nodes in dependency order and persist node-level status.

FR-05 Triggers

Manual trigger, webhook trigger and timezone-aware scheduled trigger.

FR-06 Expressions

Reference outputs from earlier nodes through a controlled expression language.

FR-07 Reliability

Retries, timeouts, cancellation, failure policy and idempotency for external side effects.

FR-08 Credentials

Encrypt secrets at rest, reference credentials by ID and redact them from logs.

4. Non-Functional Requirements

The API must remain responsive during long workflows; workers must scale independently; critical state must be durable; all external input must be validated; secrets must be protected; the system must be reproducible through Docker.

5. Technology Stack

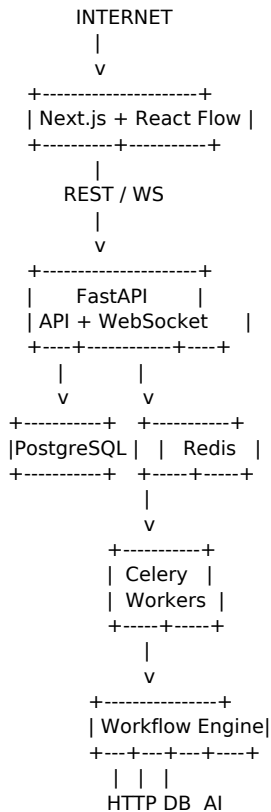
Layer	Technology	Purpose
Frontend	Next.js / React / TypeScript	Dashboard + editor
Canvas	React Flow / @xyflow/react	Node graph
UI	Tailwind + shadcn/ui	Interface
Client state	Zustand	Editor state
Backend	FastAPI + Python	REST + WebSockets
Validation	Pydantic v2	Schemas
ORM	SQLAlchemy 2.0	Database access
Migrations	Alembic	Schema migrations
DB	PostgreSQL	Durable state
Queue	Celery	Background jobs

Layer	Technology	Purpose
Broker	Redis	Queue transport
Scheduler	Celery Beat	Scheduled dispatch
Testing	Pytest + Playwright	Automated testing
Deployment	Docker	Reproducible deployment

FastAPI provides typed API development, automatic API documentation and WebSocket support.

[\[cite\]turn0search0](#)

6. High-Level Architecture



Request lifecycle

Browser -> FastAPI

- > validate request
- > create execution row
- > enqueue Celery task
- > return execution_id
- > Celery worker runs workflow
- > PostgreSQL stores results
- > WebSocket sends status
- > browser updates UI

7. Repository Structure

```
orchestrix/  
├── apps/  
│   ├── web/  
│   │   ├── app/  
│   │   ├── components/  
│   │   ├── features/  
│   │   ├── workflow-editor/  
│   │   ├── store/  
│   │   └── lib/  
│   └── backend/  
│       ├── app/  
│       │   ├── main.py  
│       │   ├── api/v1/  
│       │   ├── core/  
│       │   ├── db/  
│       │   ├── models/  
│       │   ├── schemas/  
│       │   ├── services/  
│       │   ├── workflow_engine/  
│       │   ├── nodes/  
│       │   ├── workers/  
│       │   ├── queue/  
│       │   ├── integrations/  
│       │   └── websocket/  
│       └── tests/  
└── infrastructure/
```

```
|— scripts/  
|— docker-compose.yml  
|— .env.example  
|— README.md
```

Backend modules

api: HTTP transport; core: configuration/security; db: SQLAlchemy/Alembic; models: persistence; schemas: Pydantic; services: business logic; workflow_engine: graph execution; nodes: executable node implementations; workers: Celery tasks; queue: Redis/Celery configuration; integrations: external APIs; websocket: real-time events.

8. Workflow Graph Model

```
{
  "id": "wf_123",
  "name": "Lead Automation",
  "version": 3,
  "status": "active",
  "nodes": [
    {
      "id": "trigger_1",
      "type": "webhook",
      "position": {"x": 100, "y": 200},
      "config": {}
    },
    {
      "id": "http_1",
      "type": "http-request",
      "position": {"x": 400, "y": 200},
      "config": {
        "method": "GET",
        "url": "https://api.example.com/leads/{{trigger_1.body.id}}"
      },
      "credentialId": "cred_456"
    }
  ],
  "connections": [
    {
      "source": "trigger_1",
      "sourcePort": "main",
      "target": "http_1",
      "targetPort": "main"
    }
  ]
}
```

Credentials must never be embedded as plaintext in workflow JSON.

9. PostgreSQL Schema

Table	Purpose
users	Identity
workspaces	Tenant/workspace boundary
workspace_members	RBAC
workflows	Workflow metadata
workflow_versions	Immutable workflow definitions
executions	Execution history
execution_nodes	Per-node status/input/output/error
credentials	Encrypted secret payloads
webhooks	Inbound triggers
schedules	Cron schedules
templates	Reusable workflow definitions
api_keys	Programmatic access
audit_logs	Security/activity history

Important indexes: workflows(workspace_id, updated_at), workflow_versions(workflow_id, version), executions(workflow_id, started_at), execution_nodes(execution_id), webhooks(path, method), audit_logs(workspace_id, created_at).

10. FastAPI Application Architecture

```
from fastapi import FastAPI
```

```
app = FastAPI(title="Orchestrix API", version="1.0.0")

app.include_router(auth_router, prefix="/api/v1/auth")
app.include_router(workflow_router, prefix="/api/v1/workflows")
app.include_router(execution_router, prefix="/api/v1/executions")
app.include_router(webhook_router, prefix="/api/v1/webhooks")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Keep route functions thin. Route -> dependency/auth -> service -> repository/database. Execution requests additionally enqueue Celery work.

11. API Endpoints

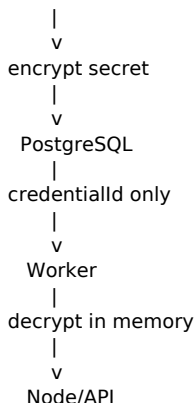
Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/logout	Logout
GET	/api/v1/workflows	List
POST	/api/v1/workflows	Create
GET	/api/v1/workflows/{id}	Get
PUT	/api/v1/workflows/{id}	Update
POST	/api/v1/workflows/{id}/activate	Activate
POST	/api/v1/workflows/{id}/execute	Execute
GET	/api/v1/executions/{id}	Details
POST	/api/v1/executions/{id}/cancel	Cancel
GET	/api/v1/credentials	List metadata
POST	/api/v1/credentials	Create
POST	/api/v1/webhooks/{workflow_id}	Trigger
GET	/api/v1/templates	Templates

12. Authentication and Authorization

Use authenticated FastAPI dependencies and enforce workspace membership before resource access. Suggested roles: Owner, Admin, Editor and Viewer. Every workflow, execution and credential query must be scoped to the current workspace.

13. Credential Management

Client -> TLS -> FastAPI -> validate



Use authenticated encryption such as AES-GCM or a managed secrets service. Keep encryption keys outside the database and redact secret values from logs.

14. Workflow Execution Engine

Lifecycle: QUEUED -> RUNNING -> SUCCEEDED/FAILED/CANCELED/TIMED_OUT. Each node can be PENDING -> RUNNING -> SUCCESS/FAILED/SKIPPED.

15. Python Workflow Engine

```
class WorkflowEngine:
    async def execute(self, workflow, trigger_data):
        graph = self.validator.validate(workflow)
        ctx = ExecutionContext(trigger_data=trigger_data)
        ready = graph.initial_nodes()

        while ready:
            node = ready.pop()
            result = await self.executor.run(node, ctx)
            ctx.store(node.id, result)
            ready.extend(graph.next_nodes(node, result))

        return ctx
```

The engine must validate duplicate node IDs, missing targets, invalid ports, unreachable required nodes, illegal cycles where unsupported, missing configuration and incompatible connections.

16. Node Architecture

```
from pydantic import BaseModel

class HttpConfig(BaseModel):
    method: str
    url: str
    headers: dict[str, str] = {}
    body: dict | None = None

class HttpRequestNode:
    type = "http-request"

    async def execute(self, ctx, config: HttpConfig):
        url = ctx.resolve(config.url)
        response = await ctx.http.request(
            config.method, url,
            headers=config.headers,
            json=config.body
        )
        return {"main": [response.json()]}
```

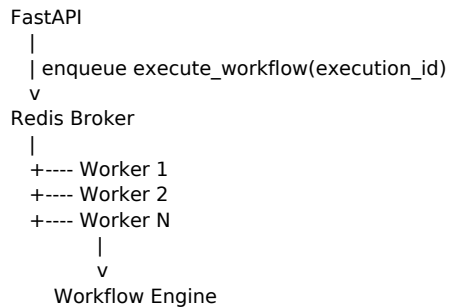
Initial nodes: Manual Trigger, Webhook, Schedule, HTTP Request, Set, IF, Switch, Delay, PostgreSQL, Slack and Email. Add AI nodes after deterministic execution is stable.

17. Expression Engine

```
{{ $json.email }}
{{ $node.Webhook.body.customerId }}
{{ $node.HTTP_Request.body.amount }}
{{ $vars.environment }}
```

Expressions should use a controlled parser/AST and allowlisted functions. Never expose arbitrary Python, filesystem, process environment or unrestricted network APIs through expressions.

18. Celery + Redis Architecture



Celery is a Python distributed task queue. Redis can act as its broker/result backend. Durable business state should remain in PostgreSQL. [\[cite\]turn0search1\[turn0search2\]](#)

```
@celery_app.task(
    bind=True,
    autoretry_for=(TransientError,),
    retry_backoff=True,
    max_retries=3
)
def execute_workflow(self, execution_id):
    return run_execution(execution_id)
```

Use bounded concurrency, retries and graceful shutdown. Do not let the API process perform long workflow execution.

19. Triggers

```
POST /api/v1/webhooks/{workflow_id}
|
v
validate -> create execution -> Celery -> worker
```

Schedules should store cron expression and timezone. The scheduler dispatches jobs; it does not execute the workflow itself.

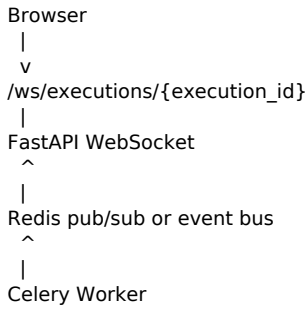
20. React Flow Frontend

```
workflow-editor/
├── canvas/
├── nodes/
├── edges/
├── config-panel/
├── toolbar/
└── execution-overlay/
```

The editor manages draft graph state, node configuration, validation, save/load, test execution and live execution status. React Flow supplies the core graph interaction primitives.

[\[cite\]turn0search3\[turn0search4\]](#)

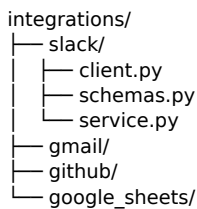
21. WebSocket Execution Updates



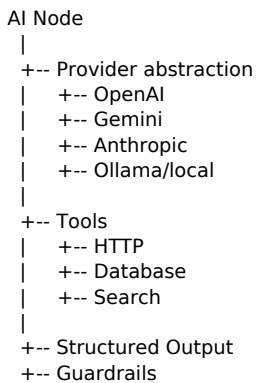
Events: execution.started, node.started, node.completed, node.failed, execution.completed and execution.failed. Include executionId, nodeId, timestamp and safe diagnostic metadata.

22. Integrations Architecture

Start with HTTP Request because it unlocks arbitrary REST APIs. Dedicated adapters should contain credential schema, client, configuration, response mapping, rate-limit behavior, retry classification and secret redaction.



23. AI and Agent Architecture



Agent controls: maximum iterations, timeout, allowed tools, structured output schema, optional human approval and usage/cost metadata.

24. Error Handling, Retries and Idempotency

Failure	Example	Policy
Validation	Missing config	No retry
Authentication	Expired token	Refresh if supported
Rate limit	HTTP 429	Backoff + retry
Timeout	Network timeout	Bounded retry
Provider 5xx	Service unavailable	Bounded retry
Provider 4xx	Bad request	Usually fail
Internal	Unexpected exception	Fail + alert

Use exponential backoff with jitter. For external side effects, use an idempotency key derived from execution, node and logical operation. Never blindly retry a non-idempotent operation.

25. Security

Security controls: TLS; secure password hashing; protected sessions/tokens; workspace authorization; rate limiting; strict CORS; secret redaction; audit logs; Pydantic validation; secure headers.

SSRF protection is mandatory for a user-configurable HTTP node. Consider blocking private/loopback ranges, checking resolved destination IPs, enforcing response-size/time limits and using an egress proxy.

Code nodes must not execute arbitrary user Python inside the main FastAPI/Celery process. A real Code node requires sandbox isolation with CPU, memory, filesystem and network restrictions.

26. Observability

Logs: application and worker errors. Metrics: queue depth, execution count, duration, failure rate and worker utilization. Traces: request -> queue -> worker -> node. Audit logs: workflow activation, deletion, credential changes and membership changes. Use an execution/correlation ID everywhere.

27. Testing Strategy

Test type	Coverage
Unit	Graph traversal, expressions, validators, node logic
API	Authentication, CRUD, authorization, webhooks
Integration	PostgreSQL, Redis/Celery, integration adapters
Worker	Task lifecycle, retry, timeout, cancellation
E2E	Create -> activate -> trigger -> inspect
Security	Authorization bypass, SSRF, secret leakage
Load	Concurrent webhook and execution traffic

Minimum acceptance workflow

Webhook -> HTTP Request -> IF -> PostgreSQL/Slack. The test must verify queueing, worker execution, persisted history, live UI updates and failure handling.

28. Docker Deployment

```
docker compose
├── postgres
├── redis
├── backend
├── celery-worker
├── celery-beat
└── web
```

Production: load balancer/reverse proxy -> Next.js + multiple FastAPI instances; Redis -> multiple Celery workers; PostgreSQL for durable state; object storage for large artifacts.

29. Environment Variables

```
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/orchestrix
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1
CELERY_RESULT_BACKEND=redis://localhost:6379/2
SECRET_KEY=replace-with-long-random-secret
CREDENTIAL_ENCRYPTION_KEY=replace-with-strong-key
FRONTEND_URL=http://localhost:3000
API_URL=http://localhost:8000
OPENAI_API_KEY=
GEMINI_API_KEY=
ANTHROPIC_API_KEY=
```

30. Local Development

```
git clone <repository>
cd orchestrix
```

```
# Backend
cd apps/backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

```
# Worker
celery -A app.workers.celery_app worker --loglevel=info --concurrency=4
```

```
# Scheduler
celery -A app.workers.celery_app beat --loglevel=info
```

```
# Frontend
cd apps/web
pnpm install
pnpm dev
```

Use Docker Compose for PostgreSQL and Redis. Commit only .env.example, never real secrets.

31. Implementation Roadmap

Phase	Deliverables
1	FastAPI, PostgreSQL, SQLAlchemy, Alembic, Redis, Docker
2	Auth and workspace model
3	Workflow CRUD and versions
4	React Flow editor
5	Python workflow engine
6	Celery workers and execution history
7	Manual/webhook/schedule triggers
8	WebSocket execution UI
9	Credentials and security
10	HTTP/PostgreSQL/Slack integrations
11	Testing, monitoring and production Docker
12	LLM node, agents and RAG

Do not begin with AI agents. First prove deterministic workflows execute reliably and recover correctly.

32. Performance and Scaling

Keep API handlers short, paginate history, index PostgreSQL queries, store large files in object storage, control Celery concurrency, use per-integration rate limits and scale API and workers independently.

MVP:

1 FastAPI + 1 Worker

|

Scale:

1 FastAPI + N Workers

|

Production:

N FastAPI + N Workers

|

Managed PostgreSQL + Redis + Object Storage

33. Backup and Recovery

Asset	Strategy
PostgreSQL	Snapshots + point-in-time recovery
Redis	Recoverable transient queue; not business source of truth
Workflows	Versioned PostgreSQL + optional exports
Credentials	Encrypted DB backups + secure key backup
Object storage	Versioning/retention
Infrastructure	Version-controlled Docker/IaC

34. Future Features

OAuth integrations, marketplace/templates, sub-workflows, human approvals, collaboration, workflow version diff, enterprise SSO, usage metering, AI workflow generation, AI debugging, RAG/vector database nodes and multi-region execution.

35. Definition of Done

The MVP is complete when authentication, workflow editing, graph persistence, workflow versions, manual/webhook execution, Celery workers, PostgreSQL execution history, WebSocket updates, encrypted credentials, retries/timeouts, at least one real integration, automated tests and Docker-based setup all work end-to-end.

Final demo flow

Create Workflow

|

Webhook -> HTTP -> IF -> PostgreSQL/Slack

|

Activate

|

Send Webhook

|

FastAPI -> Redis -> Celery Worker

|

Workflow Engine

|

Execution History + Live Status + Errors

Technical references

FastAPI documentation: API framework capabilities, automatic documentation and WebSockets.

□cite□turn0search0□

Celery documentation: distributed task queue and worker architecture. [\[cite\]](#)[\[turn0search1\]](#)[\[turn0search2\]](#)

React Flow documentation: node/edge graph editor and interaction primitives.

[\[cite\]](#)[\[turn0search3\]](#)[\[turn0search4\]](#)

This is an independent Orchestrix architecture document; it is not an official n8n document and does not reproduce n8n source code.

END OF DOCUMENT